

Тест-кейсы: Product Showcase API

Базовые проверки по всем эндпоинтам сервиса (см. Swagger UI: /docs). Формат: happy path, авторизация, валидация, граничные и сквозные проверки.

Демо-пользователь: admin / admin123 (переопределяется переменной окружения DEMO_USER_PASSWORD).

1. POST /auth/login — получение JWT-токена

ID	Название	Предусловия	Шаги	Ожидаемый результат	Приоритет
TC-AUTH-01	Успешный логин с валидными данными	Сервис запущен, тестовые данные (products.json) в исходном/известном состоянии	1. POST /auth/login с телом {username: 'admin', password: 'admin123'}	Код 200. В теле есть непустой access_token (строка) и token_type == 'bearer'	High
TC-AUTH-02	Логин с неверным паролем	Сервис запущен, тестовые данные (products.json) в исходном/известном состоянии	1. POST /auth/login с телом {username: 'admin', password: 'wrong-password'}	Код 401. Тело: code == 'INVALID_CREDENTIALS'	High
TC-AUTH-03	Логин под несуществующим пользователем	Сервис запущен, тестовые данные (products.json) в исходном/известном состоянии	1. POST /auth/login с телом {username: 'no-such-user', password: 'admin123'}	Код 401. Тело: code == 'INVALID_CREDENTIALS' (сообщение не должно раскрывать, что именно неверно — логин или пароль)	Medium
TC-AUTH-04	Логин без поля password	Сервис запущен, тестовые данные (products.json) в исходном/известном состоянии	1. POST /auth/login с телом {username: 'admin'} (без password)	Код 400. code == 'MISSING_FIELD', field == 'password'	Medium
TC-AUTH-05	Логин без поля username	Сервис запущен, тестовые данные (products.json) в исходном/известном состоянии	1. POST /auth/login с телом {password: 'admin123'} (без username)	Код 400. code == 'MISSING_FIELD', field == 'username'	Medium
TC-AUTH-06	Логин с неизвестным полем в теле	Сервис запущен, тестовые данные (products.json) в исходном/известном состоянии	1. POST /auth/login с телом {username: 'admin', password: 'admin123', extra: '1'}	Код 400. code == 'UNKNOWN_FIELD' (лишние поля запрещены)	Low
TC-AUTH-07	Логин с пустой строкой в password	Сервис запущен, тестовые данные (products.json) в исходном/известном состоянии	1. POST /auth/login с телом {username: 'admin', password: ''}	Код 401. code == 'INVALID_CREDENTIALS' (пустой пароль — не то же самое, что отсутствующее поле)	Low

2. GET /products — список товаров

ID	Название	Предусловия	Шаги	Ожидаемый результат	Приоритет
TC-LIST-01	Получение списка без параметров	В хранилище есть хотя бы 1 товар	1. GET /products	Код 200. Тело — JSON-массив. Размер массива <= 20 (лимит по умолчанию)	High
TC-LIST-02	Фильтр по min_price	В хранилище есть товары с разными ценами	1. GET /products?min_price=100	Код 200. У каждого товара в ответе price >= 100	High
TC-LIST-03	Фильтр по max_price	В хранилище есть товары с разными ценами	1. GET /products?max_price=500	Код 200. У каждого товара в ответе price <= 500	High
TC-LIST-04	Комбинация min_price и max_price	В хранилище есть товары с разными ценами	1. GET /products?min_price=100&max_price=500	Код 200. У каждого товара price в диапазоне [100, 500]	Medium
TC-LIST-05	Сортировка по возрастанию цены	В хранилище есть >= 2 товара с разными ценами	1. GET /products?sort=price_asc	Код 200. Цены в массиве идут по неубыванию	High
TC-LIST-06	Сортировка по убыванию цены	В хранилище есть >= 2 товара с разными ценами	1. GET /products?sort=price_desc	Код 200. Цены в массиве идут по невозрастанию	Medium
TC-LIST-07	Сортировка по имени А-Я	В хранилище есть >= 2 товара с разными именами	1. GET /products?sort=name_asc	Код 200. Имена идут в алфавитном порядке (неубывание)	Medium
TC-LIST-08	Сортировка по имени Я-А	В хранилище есть >= 2 товара с разными именами	1. GET /products?sort=name_desc	Код 200. Имена идут в обратном алфавитном порядке	Low
TC-LIST-09	Пагинация — параметр limit	В хранилище есть >= 5 товаров	1. GET /products?limit=2	Код 200. В ответе не больше 2 элементов	High
TC-LIST-10	Пагинация — параметр offset	В хранилище есть >= 5 товаров, известен порядок без сортировки	1. GET /products?offset=2&limit=2 2. Сравнить с полным списком GET /products	Код 200. Возвращены элементы с индекса 2 и 3 исходного списка	Medium
TC-LIST-11	limit меньше минимума (limit=0)	Сервис запущен, тестовые данные (products.json) в исходном/известном состоянии	1. GET /products?limit=0	Код 400. code == 'VALIDATION_ERROR', field == 'limit' (ограничение ge=1)	Medium
TC-LIST-12	limit больше максимума (limit=101)	Сервис запущен, тестовые данные (products.json) в исходном/известном состоянии	1. GET /products?limit=101	Код 400. code == 'VALIDATION_ERROR', field == 'limit' (ограничение le=100)	Medium

ID	Название	Предусловия	Шаги	Ожидаемый результат	Приоритет
TC-LIST-13	offset отрицательный	Сервис запущен, тестовые данные (products.json) в исходном/известном состоянии	1. GET /products?offset=-1	Код 400. code == 'VALIDATION_ERROR', field == 'offset' (ограничение ge=0)	Medium
TC-LIST-14	Некорректное значение sort	Сервис запущен, тестовые данные (products.json) в исходном/известном состоянии	1. GET /products?sort=invalid_value	Код 400. code == 'VALIDATION_ERROR', field == 'sort'	Medium
TC-LIST-15	min_price не является числом	Сервис запущен, тестовые данные (products.json) в исходном/известном состоянии	1. GET /products?min_price=abc	Код 400. code == 'VALIDATION_ERROR', field == 'min_price'	Low
TC-LIST-16	Фильтр не находит ни одного товара	В хранилище нет товаров дороже 1 000 000	1. GET /products?min_price=1000000	Код 200. Тело — пустой массив []	Low
TC-LIST-17	Порядок применения: фильтр -> сортировка -> пагинация	В хранилище есть >= 5 товаров с разными ценами	1. GET /products?min_price=50&sort=price_asc&limit=2&offset=1	Код 200. Возвращены 2 товара: сначала применён фильтр по цене, потом сортировка по возрастанию, потом взят срез [1:3]	Medium

3. GET /products/{id} — товар по идентификатору

ID	Название	Предусловия	Шаги	Ожидаемый результат	Приоритет
TC-GET-01	Получение существующего товара	Товар создан заранее (например, через POST /products), известен его id	1. GET /products/{id} с id существующего товара	Код 200. Тело содержит id, name, price, description, соответствующие созданному товару	High
TC-GET-02	Получение несуществующего товара	Сервис запущен, тестовые данные (products.json) в исходном/известном состоянии	1. GET /products/{id} со случайным несуществующим uuid	Код 404. code == 'NOT_FOUND', field == 'id'	High
TC-GET-03	id в произвольном (не-uuid) формате	Сервис запущен, тестовые данные (products.json) в исходном/известном состоянии	1. GET /products/not-a-valid-id	Код 404. code == 'NOT_FOUND' (сервис не валидирует формат id, просто не находит совпадение)	Low

4. POST /products — создание товара (требуется авторизация)

ID	Название	Предусловия	Шаги	Ожидаемый результат	Приоритет
TC-CREATE-01	Успешное создание с обязательными полями	Получен валидный access-токен (POST /auth/login под admin/admin123)	1. POST /products с телом {name: 'Товар', price: 199.99}, заголовок Authorization: Bearer	Код 201. Тело содержит сгенерированный id (uuid), name и price совпадают с запросом, description == null	High
TC-CREATE-02	Успешное создание со всеми полями (включая description)	Получен валидный access-токен (POST /auth/login под admin/admin123)	1. POST /products с телом {name: 'Товар', price: 199.99, description: 'Описание'}	Код 201. Все поля в ответе совпадают с переданными, добавлен id	High
TC-CREATE-03	Создание без заголовка Authorization	Сервис запущен, тестовые данные (products.json) в исходном/известном состоянии	1. POST /products с валидным телом, без заголовка Authorization	Код 401. code == 'UNAUTHORIZED'. Товар не создан	High
TC-CREATE-04	Создание с невалидным (мусорным) токеном	Сервис запущен, тестовые данные (products.json) в исходном/известном состоянии	1. POST /products с заголовком Authorization: Bearer invalid.token.value	Код 401. code == 'UNAUTHORIZED'. Товар не создан	Medium
TC-CREATE-05	Создание с истёкшим токеном	Есть токен с истёкшим сроком действия (exp в прошлом)	1. POST /products с заголовком Authorization: Bearer	Код 401. code == 'UNAUTHORIZED'. Товар не создан	Medium
TC-CREATE-06	Отсутствует обязательное поле name	Получен валидный access-токен (POST /auth/login под admin/admin123)	1. POST /products с телом {price: 10}	Код 400. code == 'MISSING_FIELD', field == 'name'. Товар не создан	Medium
TC-CREATE-07	Отсутствует обязательное поле price	Получен валидный access-токен (POST /auth/login под admin/admin123)	1. POST /products с телом {name: 'Товар'}	Код 400. code == 'MISSING_FIELD', field == 'price'. Товар не создан	Medium
TC-CREATE-08	Передано неизвестное поле	Получен валидный access-токен (POST /auth/login под admin/admin123)	1. POST /products с телом {name: 'Товар', price: 10, sku: 'ABC'}	Код 400. code == 'UNKNOWN_FIELD'. Товар не создан	Low
TC-CREATE-09	name — пустая строка	Получен валидный access-токен (POST /auth/login под admin/admin123)	1. POST /products с телом {name: '', price: 10}	Код 400. code == 'VALIDATION_ERROR', field == 'name'	Medium

ID	Название	Предусловия	Шаги	Ожидаемый результат	Приоритет
TC-CREATE-10	name состоит только из пробелов	Получен валидный access-токен (POST /auth/login под admin/admin123)	1. POST /products с телом {name: ' ', price: 10}	Код 400. code == 'VALIDATION_ERROR', field == 'name'	Low
TC-CREATE-11	name длиннее 100 символов	Получен валидный access-токен (POST /auth/login под admin/admin123)	1. POST /products с телом {name: 'A' * 101, price: 10}	Код 400. code == 'VALIDATION_ERROR', field == 'name'	Low
TC-CREATE-12	price равен нулю	Получен валидный access-токен (POST /auth/login под admin/admin123)	1. POST /products с телом {name: 'Товар', price: 0}	Код 400. code == 'VALIDATION_ERROR', field == 'price' (цена должна быть > 0)	Medium
TC-CREATE-13	price отрицательный	Получен валидный access-токен (POST /auth/login под admin/admin123)	1. POST /products с телом {name: 'Товар', price: -5}	Код 400. code == 'VALIDATION_ERROR', field == 'price'	Medium
TC-CREATE-14	price с округлением до 2 знаков	Получен валидный access-токен (POST /auth/login под admin/admin123)	1. POST /products с телом {name: 'Товар', price: 9.995}	Код 201. price в ответе округлён до 2 знаков по правилу ROUND_HALF_UP (9.995 -> 10.0)	Low
TC-CREATE-15	price не является числом	Получен валидный access-токен (POST /auth/login под admin/admin123)	1. POST /products с телом {name: 'Товар', price: 'дорого'}	Код 400. code == 'VALIDATION_ERROR', field == 'price'	Low

5. PUT /products/{id} — полное обновление товара (требуется авторизация)

ID	Название	Предусловия	Шаги	Ожидаемый результат	Приоритет
TC-UPDATE-01	Успешное обновление существующего товара	Получен валидный access-токен (POST /auth/login под admin/admin123); товар создан заранее	1. PUT /products/{id} с телом {name: 'Новое имя', price: 50, description: 'Новое описание'}	Код 200. Тело содержит те же id, обновлённые name/price/description	High

ID	Название	Предусловия	Шаги	Ожидаемый результат	Приоритет
TC-UPDATE-02	Обновление с description = null	Получен валидный access-токен (POST /auth/login под admin/admin123); товар создан заранее	1. PUT /products/{id} с телом {name: 'Товар', price: 50, description: null}	Код 200. description в ответе == null (ключ передан явно со значением null — это допустимо)	Medium
TC-UPDATE-03	Обновление без авторизации	Товар создан заранее	1. PUT /products/{id} с валидным телом, без заголовка Authorization	Код 401. code == 'UNAUTHORIZED'. Товар не изменён	High
TC-UPDATE-04	Обновление несуществующего товара	Получен валидный access-токен (POST /auth/login под admin/admin123)	1. PUT /products/{id} со случайным несуществующим id, валидное тело	Код 404. code == 'NOT_FOUND'	High
TC-UPDATE-05	Ключ description вообще отсутствует в теле	Получен валидный access-токен (POST /auth/login под admin/admin123); товар создан заранее	1. PUT /products/{id} с телом {name: 'Товар', price: 50} (без ключа description)	Код 400. code == 'MISSING_FIELD', field == 'description' (ключ обязателен, даже если значение может быть null)	Medium
TC-UPDATE-06	Отсутствует поле name	Получен валидный access-токен (POST /auth/login под admin/admin123); товар создан заранее	1. PUT /products/{id} с телом {price: 50, description: null}	Код 400. code == 'MISSING_FIELD', field == 'name'	Medium
TC-UPDATE-07	Отсутствует поле price	Получен валидный access-токен (POST /auth/login под admin/admin123); товар создан заранее	1. PUT /products/{id} с телом {name: 'Товар', description: null}	Код 400. code == 'MISSING_FIELD', field == 'price'	Medium
TC-UPDATE-08	Передано неизвестное поле	Получен валидный access-токен (POST /auth/login под admin/admin123); товар создан заранее	1. PUT /products/{id} с телом {name: 'Товар', price: 50, description: null, sku: 'X'}	Код 400. code == 'UNKNOWN_FIELD'	Low
TC-UPDATE-09	price <= 0 при обновлении	Получен валидный access-токен (POST /auth/login под admin/admin123); товар создан заранее	1. PUT /products/{id} с телом {name: 'Товар', price: 0, description: null}	Код 400. code == 'VALIDATION_ERROR', field == 'price'	Medium

ID	Название	Предусловия	Шаги	Ожидаемый результат	Приоритет
TC-UPDATE-10	Идемпотентность — повторная отправка того же тела	Получен валидный access-токен (POST /auth/login под admin/admin123); товар создан заранее	1. PUT /products/{id} с телом X 2. Повторно PUT /products/{id} с тем же телом X	Оба ответа — код 200 с идентичным телом	Medium
TC-UPDATE-11	name пустой/только пробелы при обновлении	Получен валидный access-токен (POST /auth/login под admin/admin123); товар создан заранее	1. PUT /products/{id} с телом {name: ' ', price: 50, description: null}	Код 400. code == 'VALIDATION_ERROR', field == 'name'	Low

6. DELETE /products/{id} — удаление товара (требуется авторизация)

ID	Название	Предусловия	Шаги	Ожидаемый результат	Приоритет
TC-DELETE-01	Успешное удаление существующего товара	Получен валидный access-токен (POST /auth/login под admin/admin123); товар создан заранее	1. DELETE /products/{id}	Код 204. Тело ответа пустое	High
TC-DELETE-02	Товар действительно удалён из хранилища	Товар удалён в предыдущем шаге (TC-DELETE-01)	1. GET /products/{id} тем же id	Код 404. code == 'NOT_FOUND' — товара больше нет	High
TC-DELETE-03	Удаление без авторизации	Товар создан заранее	1. DELETE /products/{id} без заголовка Authorization	Код 401. code == 'UNAUTHORIZED'. Товар не удалён	High
TC-DELETE-04	Удаление несуществующего товара	Получен валидный access-токен (POST /auth/login под admin/admin123)	1. DELETE /products/{id} со случайным несуществующим id	Код 404. code == 'NOT_FOUND'	Medium
TC-DELETE-05	Повторное удаление уже удалённого товара	Товар только что удалён	1. DELETE /products/{id} тем же id повторно	Код 404. code == 'NOT_FOUND' (повторное удаление не идемпотентно по коду ответа — второй раз 404)	Medium

7. Сквозные проверки

ID	Название	Предусловия	Шаги	Ожидаемый результат	Приоритет
TC-GEN-01	Единый формат тела ошибки	Сервис запущен, тестовые данные (products.json) в исходном/известном состоянии	1. Вызвать любой эндпоинт так, чтобы получить 4xx/5xx (например GET /products/unknown-id)	Тело ответа содержит ровно поля code (строка), message (строка), field (строка или null)	Medium
TC-GEN-02	Content-Type ответа — application/json	Сервис запущен, тестовые данные (products.json) в исходном/известном состоянии	1. GET /products	Заголовок Content-Type ответа начинается с 'application/json'	Low
TC-GEN-03	Публичные GET-эндпоинты доступны без токена	Сервис запущен, тестовые данные (products.json) в исходном/известном состоянии	1. GET /products без заголовка Authorization 2. GET /products/{id} без заголовка Authorization	Оба запроса возвращают ожидаемые коды (200/404), а не 401 — авторизация не требуется	Medium
TC-GEN-04	Изменяющие эндпоинты защищены единообразно	Сервис запущен, тестовые данные (products.json) в исходном/известном состоянии	1. Без токена вызвать по очереди: POST /products, PUT /products/{id}, DELETE /products/{id}	Все три запроса возвращают 401 с code == 'UNAUTHORIZED'	Medium